

Tab 1

CSE / EEE / ETE 499A (Section 1)

Project Proposal (CO1)

Project Title:

ShopMate — A Catalog-Grounded, Omnichannel AI Sales & Support Agent for Facebook, Instagram, WhatsApp, and Website Commerce

Submitted To

Dr. Shazzad Hosain (SZZ)

Date: 05/07/2026



Semester: Summer 2026

Group No: G-1

Group Members

ID	Name
2311966642	Safat Uddin
2311790642	Farhan Karim
2311443042	Safwan Ismaun Amin
2311195042	Mahir Shahriyar Adib

1. Introduction

1.1 Background and State of the Art

Conversational, chat-based commerce has moved from a novelty feature to the default storefront for a large share of small and medium businesses, particularly across South and Southeast Asia. Independent market analysis places the global conversational-commerce market at roughly **USD 12.6 billion in 2026**, growing at a compound annual rate above 12%, driven almost entirely by messaging platforms Facebook Messenger, Instagram Direct, and WhatsApp Business absorbing the role that standalone web storefronts used to play [1]. WhatsApp alone now hosts more than **284 million active business accounts** and channels an estimated USD 45 billion in commerce globally in 2026 [2].

Bangladesh is a particularly acute example of this shift. The country's “f-commerce” ecosystem of tens of thousands of micro and small merchants who sell exclusively through a Facebook Page or Instagram profile rather than a dedicated website depends entirely on manual, human-answered Messenger and WhatsApp threads. This is precisely the market segment that products such as *LazyChat*, *ManyChat*, *Wati*, *Tidio*, and *Haptik* have built commercial businesses around: an AI layer that sits on top of a merchant's existing Facebook Page, Instagram account, WhatsApp Business number, and storefront, and answers customers automatically instead of routing everything to an overworked human agent.

The underlying AI technique that makes this viable at commerce scale is **Retrieval-Augmented Generation (RAG)**: rather than letting a large language model answer purely from its parametric memory (which risks fabricated prices, stock levels, or delivery promises), the model's answer is grounded in facts retrieved from the merchant's own product catalog at request time. Recent work confirms both the necessity and the limits of this approach: RAG measurably reduces hallucination and improves out-of-domain generalization in structured business-output tasks [3], yet retrieval-augmented systems can still hallucinate if the retrieval step itself is weak or the grounding is not enforced as a hard constraint [4], and dedicated studies on e-commerce product Q&A show the same pattern: RAG pipelines that constrain generation strictly to retrieved product data measurably outperform free-generation baselines on factual accuracy [5]. This directly motivates a core design rule for this project: the AI must never state a price, stock level, or delivery promise that is not backed by retrieved, tenant-specific catalog data.

A second, Bangladesh-specific technical challenge is language. Customers write in Bangla, English, and “Banglish” (romanized, code-mixed Bangla) interchangeably, sometimes within a single sentence. This is an active NLP research area in its own right: labelled code-mixed corpora such as *SentMix-3L* (Bangla-English-Hindi) [6] and *BanglishRev*, a large-scale Bangla-English code-mixed e-commerce review dataset [7], exist specifically because standard sentiment and intent models under-perform on this mixed register, and prior work on Bangla-English code-mixed social media sentiment confirms that transformer-based models substantially outperform classical machine-learning baselines on this task [8].

1.2 Problem Statement

Merchants who sell through Facebook, Instagram, WhatsApp, and a website simultaneously face three compounding problems:

- **Fragmented inboxes and lost sales** — messages arrive across four disconnected channels; a slower reply than a competitor's often means a lost customer.
- **Repetitive, high-volume Tier-1 load** — the overwhelming majority of inbound messages are the same handful of questions (price, stock, size/color, delivery time), which still consume disproportionate staff time.

- **Generic tools that are not trustworthy for commerce** — many existing chatbot builders either free-generate answers (risking a wrong price or a promise the merchant cannot keep) or do not support Bangla/Banglish well enough to be usable for the Bangladeshi SME segment specifically.

1.3 Objective of the Proposed Project

The objective of this capstone project is to design and build **ShopMate**, a multi-tenant, catalog-grounded, omnichannel AI sales-and-support agent that:

- Unifies Facebook Messenger, Instagram DM/comments, WhatsApp Business, and a website chat widget into a single conversation model and a single merchant-facing inbox.
- Generates replies that are grounded in the merchant's own product catalog (RAG), in Bangla, English, or Banglish depending on how the customer writes, rather than free-generating potentially inaccurate answers.
- Detects purchase intent and negative sentiment/complaints, builds an in-chat cart, creates an order record, and routes complaint threads to a dedicated human-follow-up queue.
- Meters every AI-generated reply per tenant (for future billing) and exposes a merchant dashboard for inbox, catalog, orders, and basic analytics.

1.4 Adopted Approach

The system is designed around five architectural principles multi-tenancy from day one, channel adapters instead of channel-coupling, catalog-grounded AI instead of open generation, treating all inbound webhooks as bursty and untrusted, and metering everything that touches AI generation and is built as an **event-driven microservice system** rather than a monolith, so that channel adapters, the AI orchestration layer, and the commerce engine can be developed, deployed, and scaled independently. Given the fixed seven-month window spanning CSE499A and CSE499B, the build is intentionally phased: CSE499A delivers the architecture, the catalog-grounded AI pipeline, and a working single/dual-channel MVP (website widget + Facebook Messenger); CSE499B extends this into the full omnichannel product WhatsApp and Instagram, the order/cart engine, complaint routing, usage metering, and a polished merchant dashboard followed by integration testing and evaluation. Full details of the phase split are given in Section 4.

2. Literature Review / Existing Projects

This section reviews (a) the AI/NLP techniques the project depends on, and (b) commercial systems already operating in this space, both internationally and in the local Bangladeshi market.

2.1 Retrieval-Augmented Generation for Factual Grounding

Béchar and Ayala (2024) show that coupling a small, well-trained retriever with an LLM significantly reduces hallucination and improves generalization on structured, enterprise-style output tasks, and that this can be done without a very large accompanying model an important cost consideration for a student-built, resource-constrained system [3]. Huang *et al.*'s survey of hallucination in large language models specifically documents that RAG systems remain vulnerable to hallucination when the retrieved context is incomplete or the generation step is not tightly constrained to it, which is the justification for treating catalog-grounding as a *hard rule* (Section 10.3) rather than a soft preference in this project's design [4]. Bergvall's applied study on RAG for e-commerce product Q&A directly parallels this project's core use case, constraining a retriever-plus-LLM pipeline to only use verified product data and reporting materially lower hallucination rates than a pure-generation baseline [5].

2.2 Bangla / Bangla-English Code-Mixed NLP

Raihan *et al.* introduce SentMix-3L, a gold-labelled Bangla-Hindi-English code-mixed sentiment dataset, and find that zero-shot prompting with a general-purpose LLM outperforms fine-tuned transformer baselines on this mixed-language setting — a useful data point favoring an LLM-first approach over training a bespoke classifier from scratch for language/intent detection in this project [6]. The BanglishRev dataset provides a large-scale, e-commerce-specific Bangla-English code-mixed review corpus and reports that a model trained on it reaches 94% accuracy on sentiment classification, confirming that Bangla/Banglish e-commerce sentiment is a tractable, well-defined task rather than an open research problem [7]. Complementary work on Bangla-English code-mixed social media sentiment (with emoji handling) confirms that transformer-based approaches consistently outperform classical machine-learning pipelines (Naïve Bayes, SVM) on this register, reinforcing the choice of a transformer/LLM-based classifier over rule-based sentiment logic for the complaint-detection module in Section 6.4 [8].

2.3 Comparable Commercial Systems

Several commercial products already operate in the AI-omnichannel-commerce space and serve as the primary benchmark for this project's feature and architecture decisions:

- **LazyChat** (platform.lazychat.io) — the direct case study for this project. It markets itself as an “AI Sales Agent for Facebook, Instagram & WhatsApp,” with image-based product recognition, native Bangla/English/Banglish replies, automatic complaint routing, an all-in-one inbox, in-chat order/cart creation, and Shopify/WooCommerce catalog sync — the feature set this project's requirements were reverse-engineered from.
- **ManyChat and Tidio** — general-purpose chat-automation/marketing platforms with broad channel support but comparatively shallow commerce-catalog grounding and no dedicated Bangla/Banglish handling.
- **Wati and Haptik** — WhatsApp-API-first business messaging platforms, strong on the WhatsApp Business Platform / Meta Embedded Signup flow specifically, but built primarily for enterprise CX rather than SME-priced, catalog-grounded sales automation.

No prior academic capstone project at this scale (multi-channel, catalog-grounded, Bangla-aware, metered SaaS architecture) was identified in the local literature; this project's contribution is therefore an integrated, reproducible engineering blueprint rather than a novel algorithm — consistent with the CO1 expectation of illustrating and systematically solving an engineering/computing problem under real constraints.

3. Business Viability of the Project

3.1 Novelty

ShopMate's requirements were reverse-engineered from LazyChat and comparable products (Section 2.3), so its novelty is not a new feature set but an architectural one: LazyChat and similar tools are closed commercial systems with no published engineering account of how catalog-grounding, complaint routing, or multilingual handling are actually implemented. ShopMate's contribution is (a) catalog-grounded generation enforced as a hard constraint at the architecture level — verified against retrieved data rather than left to a prompt instruction — and (b) a reproducible, documented engineering blueprint for building this class of system under real resource constraints, which does not currently exist in the public or academic literature at this scale

3.2 Market Segments

- **Primary:** Bangladeshi small and medium f-commerce merchants selling via Facebook Page, Instagram, and/or WhatsApp Business, currently relying on manual replies.
- **Secondary:** Broader South and Southeast Asian social-commerce SMEs facing the same fragmented-inbox problem, once multilingual support generalizes beyond Bangla.

3.3 Competitor Analysis

Product	Bangla / Banglish	Catalog-Grounded (RAG)	Complaint Routing	Target Market
ShopMate (proposed)	Native, LLM-based	Hard rule, per-tenant RAG	Automatic + human queue	BD/South Asian SMEs
LazyChat	Yes (advertised)	Implied (catalog sync)	Yes (advertised)	BD f-commerce SMEs
ManyChat	Partial (generic NLU)	Limited	Manual tagging	Global, broad SMB
Wati / Haptik	Partial	Limited (WA-first)	Manual escalation	Enterprise CX

3.4 Competitive Advantages

- Hard grounding rule eliminates a major class of trust-breaking errors (wrong price/stock quoted to a customer) that generic chatbot builders are exposed to.
- Bangla/Banglish handling is a first-class design requirement, not an afterthought, matching the actual language behavior of the target market.
- Multi-tenant, metered architecture designed from day one for SaaS-style per-tenant billing, avoiding a costly later rewrite.

3.5 Success Factors

Must-haves (minimum viable outcome for CSE499B):

- Working Facebook Messenger and WhatsApp adapters with signature-verified, deduplicated webhook ingestion.
- Catalog-grounded RAG pipeline that answers price/stock/variant questions correctly from a synced product catalog.
- Bangla / English / Banglish language detection and matched-language reply generation.
- Complaint detection and routing into a dedicated queue.
- In-chat cart and order creation, visible on a merchant order-tracking dashboard.
- Per-tenant AI-message usage metering.

Nice-to-haves (stretch goals, time-permitting in CSE499B):

- Instagram DM/comment automation and Shopify/WooCommerce live sync.
- Image-based product recognition from a customer-sent photo.
- Suggestive AI / upsell recommendations.
- A lightweight mobile companion view.

Success will be measured by: reply accuracy against the seeded test catalog (target $\geq 90\%$ on price/stock questions), correct complaint routing (recall on a labelled test set), successful end-to-end order creation from a chat conversation, and measured average response latency versus a manual baseline.

3.6 Risk Factors

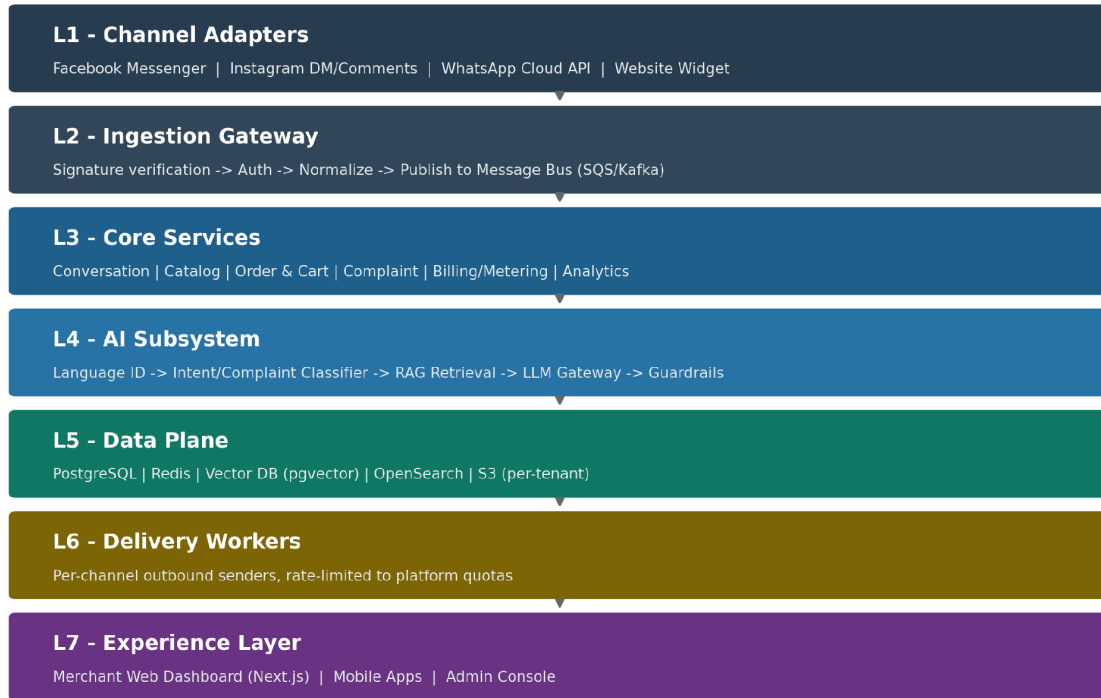
- **Platform policy risk** — Meta/WhatsApp Business Platform compliance (business verification, 24-hour messaging window, template approval) is a hard external dependency outside the team's control and can block a channel entirely if mishandled.
- **Hallucination/liability risk** — an ungrounded AI answer that quotes a wrong price or promise is a real commerce-trust failure, not a cosmetic bug; mitigated by the hard grounding rule and an escalate-on-uncertainty default (Section 6.6, 10.3).
- **Cost risk** — LLM API usage is metered per message; a bug or retry storm could generate unexpectedly high inference cost during testing, mitigated with per-tenant rate limits and a tiered cheap/expensive model strategy.
- **Data privacy risk** — the system handles real customer PII (names, phone numbers, order data) even in a prototype; mitigated by using seeded synthetic merchant/customer data for all demos and evaluation, never live customer data.
- **Schedule risk** — seven months across two courses is tight for a five-layer, multi-channel system; mitigated by the strict MVP-first phase split in Section 4 and by explicitly deferring “nice-to-have” features to a stretch backlog.

4. Project Analysis

4.1 Major Components of the Project

The system is organized into seven layers, from channel-facing edge adapters down to the merchant-facing dashboard. The figure below summarizes the layered architecture.

ShopMate AI - Layered System Architecture



- **Edge / Channel Adapters** — Meta webhook receiver (Messenger, Instagram DM/comments), WhatsApp Cloud API receiver, website chat widget.
- **Ingestion Gateway** — signature verification, auth, request shaping, and publishing normalized events onto a message bus.
- **Core Services** — Conversation, Catalog, Order & Cart, Complaint/Ticketing, Notification, Billing & Metering, and Analytics services.
- **AI Subsystem** — LLM Gateway, language/intent detection, RAG retrieval engine, and a guardrails/safety filter.
- **Data Plane** — PostgreSQL (transactional), Redis (cache/session), pgvector (embeddings), OpenSearch (search/analytics), S3 (per-tenant media).
- **Delivery Workers** — per-channel outbound senders respecting each platform's rate limits and messaging-window rules.
- **Experience Layer** — merchant web dashboard (inbox, catalog, orders, billing, analytics) and, as a stretch goal, a lightweight mobile view.

4.2 Target for CSE499A and CSE499B

CSE499A target: a working, catalog-grounded AI reply pipeline on at least one live channel (website widget) plus one Meta channel (Facebook Messenger), with the unified message contract, catalog service, RAG retrieval, and LLM Gateway all functioning end-to-end against a seeded demo merchant catalog.

CSE499B target: full omnichannel coverage (adding WhatsApp and Instagram), the order/cart engine, complaint detection and routing, usage metering, and a merchant dashboard followed by integration testing, evaluation against the success metrics in Section 3.5, and final documentation.

4.3 Task Analysis, Activity Flow, and Critical Path

The critical path runs through the components that every later feature depends on, in this order:

- 1. Unified Message Contract (channel-agnostic event schema) — blocks every adapter and the Conversation Service.
- 2. Webhook Ingestion Gateway (signature verification, dedup, enqueue) — blocks all live-channel testing.
- 3. Catalog Service + embeddings index — blocks the RAG retrieval step and therefore all grounded replies.
- 4. LLM Gateway + RAG retrieval — blocks reply generation, which blocks the CSE499A demo milestone.
- 5. Order & Cart Service — depends on (3) and (4); blocks the order-tracking dashboard.
- 6. Complaint classifier + routing — depends on (1)–(2); can be built in parallel with (5).
- 7. Metering + Billing Service — depends on every service that emits a usage_events row; scheduled last since it is additive, not blocking.

Items 1–4 form the true critical path for the CSE499A milestone; items 5–7 are largely parallelizable across team members once the message contract and catalog service exist, which is why they are the first two deliverables in Month 1–2 of the timeline below.

4.4 Timeline (7 Months, CSE499A + CSE499B)

Month	Course	Milestone
Month 1 (July)	CSE499A	Requirements freeze, architecture diagram, tech-stack setup, repo/CI scaffolding, sandbox Meta developer app.
Month 2 (August)	CSE499A	Unified message contract; website widget + Facebook Messenger adapters; webhook ingestion gateway.
Month 3 (September)	CSE499A	Catalog Service + product embeddings; RAG retrieval pipeline; LLM Gateway (single provider); end-to-end MVP demo on website widget + Messenger; CO1 defense.
Month 4 (October)	CSE499B	End-to-end MVP demo: catalog-grounded replies on 1–2 channels; CO1/CO2 progress report and defense.
Month 5 (November)	CSE499B	WhatsApp Cloud API + Instagram adapter scaffolding begins; architecture handoff from CSE499A.
Month 6 (December)	CSE499B	Usage metering service; merchant dashboard (inbox, catalog, orders, billing view); guardrails/rate limiting.
Month 7 (January)	CSE499B	Integration testing, evaluation against success metrics (Section 3.5), documentation, final defense.

4.5 People Management

Work is planned around four functional roles so that the critical-path items in Section 4.3 can proceed in parallel once the message contract is fixed:

- **Backend / Infrastructure Lead** — ingestion gateway, message bus, core services, deployment (Docker/CI, database schema).

- **AI / ML Lead** — RAG pipeline, LLM Gateway, language detection, complaint/intent classifiers, guardrails.
- **Commerce & Channel Lead** — channel adapters (Messenger/WhatsApp/Instagram/widget), Catalog and Order & Cart services.
- **Frontend / QA Lead** — merchant dashboard, test-catalog seeding, integration testing, documentation, and CO deliverable write-ups.

All four roles review each other's pull requests weekly and rotate ownership of the CO1–CO4 deliverable documents so no single member is responsible for both building and grading-critical writing.

4.6 Cost Analysis

As an academic capstone prototype (not a production SaaS launch), the cost profile is dominated by free-tier and student-credit resources, with the largest variable cost being metered LLM API calls during testing:

Item	Estimate	Notes
Cloud infrastructure (AWS)	~\$0–\$50/mo	AWS Free Tier + AWS Educate/student credits during build; scaled down (single small instance, managed Postgres/Redis tier) since this is a prototype, not production traffic.
LLM API usage	~\$20–\$40/mo	Tiered strategy: cheap/fast model for language ID, intent, and complaint classification; a frontier model only for final customer-facing generation, on a small seeded test-catalog volume.
Vector DB	\$0	pgvector co-located with Postgres — no separate managed vector-DB cost at prototype scale.
Meta/WhatsApp developer sandbox	\$0	Meta for Developers test app and WhatsApp Cloud API sandbox numbers are free for development/testing.
Domain + misc.	~\$15 one-time	Optional custom domain for the demo dashboard/website widget.
Estimated total (7 months)	~\$150–\$350	Excludes team labor; assumes student/free-tier credits are used wherever available.

References

- [1] M. Intelligence, "Conversational Commerce Market Size, Share & Trends Report," Mordor Intelligence, 2026. [Online]. Available: <https://www.mordorintelligence.com/industry-reports/conversational-commerce-market>
- [2] Invent, "The \$45B WhatsApp Business Economy: How to Capture Your Share (2026 Guide)," 2026. [Online]. Available: <https://www.useinvent.com/blog/the-usd45b-whatsapp-business-economy-how-to-capture-your-share-2026-guide>
- [3] P. Béchard and O. Marquez Ayala, "Reducing hallucination in structured outputs via Retrieval-Augmented Generation," arXiv:2404.08189, 2024.
- [4] L. Huang et al., "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions," arXiv:2311.05232, 2023.
- [5] P. Bergvall, "Reducing LLM Hallucinations in Product Q&A Systems: A Retrieval-Augmented Generation Approach," SSRN, 2023. [Online]. Available: <https://ssrn.com/abstract=5223641>

- [6] M. N. Raihan, D. Goswami, A. Mahmud, A. Anastasopoulos, and M. Zampieri, "SentMix-3L: A Bangla-English-Hindi Code-Mixed Dataset for Sentiment Analysis," arXiv:2310.18023, 2023.
- [7] "BanglishRev: A Large-Scale Bangla-English and Code-mixed Dataset of Product Reviews in E-Commerce," arXiv:2412.13161, 2024.
- [8] "Sentiment Analysis of Bangla-English Code-Mixed and Transliterated Social Media Comments Using Machine Learning," Proc. NLP Tool Contest, ICON, 2023.
- [9] LazyChat, "AI Sales Agent for Facebook, Instagram & WhatsApp," lazychat.io and platform.lazychat.io [Online]. Accessed 2026.